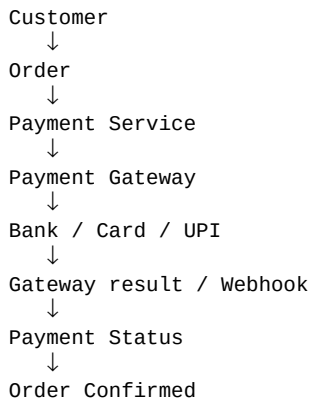


Payment System — Practical System Design Notes

Hinglish • Idempotency • Webhooks • Ledger • Reconciliation • Interview-focused

1. Payment System Kya Hai?

Payment system ka kaam customer se payment initiate karna, payment provider/gateway ke saath coordinate karna, payment state reliably maintain karna aur final business state ko correct rakhna hai.



2. Core Problem

Payment system mein network failures, retries, browser refresh, gateway timeout aur delayed webhooks normal situations hain. Isliye design ko duplicate charge aur incorrect order state se protect karna hota hai.

3. Order vs Payment

Order
→ what customer wants to buy

Payment
→ how/when money is collected

Order ID: ORD-1001
Payment ID: PAY-9001

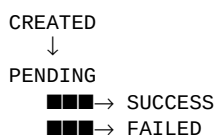
One order can have multiple payment attempts.

Order aur payment ko same entity treat karna avoid karo. Failed payment ke baad same order par another payment attempt possible ho sakta hai.

4. Basic Payment Flow

1. Create Order
2. Create Payment record
3. Generate payment intent/order at gateway
4. Customer completes payment
5. Gateway returns result or later sends webhook
6. Verify gateway information
7. Update payment state
8. Confirm order only after valid successful payment

5. Payment State Machine



■■■→ EXPIRED

SUCCESS



REFUND_PENDING



REFUNDED / PARTIALLY_REFUNDED

Exact states gateway/business requirements ke according vary kar sakte hain. Important hai ki invalid state transitions prevent kiye jayein.

6. Idempotency — Most Important

Agar customer ne Pay button ek baar click kiya aur network timeout ho gaya, client retry kar sakta hai. Server ko second request ko second charge nahi banana chahiye.

```
POST /payments
Idempotency-Key: ORDER-1001-ATTEMPT-1
```

Request 1:
→ payment created

Response lost

Request 2:
same Idempotency-Key

Server:
→ existing payment result return
→ don't create another charge

7. Idempotency Table

```
idempotency_key
user_id
order_id
payment_id
request_hash
status
response
created_at
```

Unique:
(idempotency_key, user_id)

Request payload change karke same idempotency key reuse ki jaye to system ko reject/mismatch handle karna chahiye.

8. Why Client-Side Protection Is Not Enough

Button disable karna UX improvement hai, financial guarantee nahi. Network retry, multiple devices, duplicated requests aur malicious clients possible hain. Idempotency enforcement backend par hona chahiye.

9. Payment Gateway

Gateway merchant application aur underlying payment methods/banks/networks ke beech integration layer provide karta hai. Application ko gateway APIs aur webhooks ke documented contracts follow karne chahiye.

10. Redirect / SDK / Hosted Checkout

```
Your Backend
↓
Create payment intent/order
↓
Gateway Checkout / SDK
↓
Customer pays
↓
Gateway
↓
Return / Webhook
↓
Your Backend verifies
```

Browser redirect ko final source of truth treat nahi karna chahiye. Server-side verification/webhook handling important hai.

11. Webhook

Webhook gateway ka server-to-server callback/event hota hai jo payment status change ke baare mein merchant backend ko notify karta hai.

```
Gateway
↓
POST /webhooks/payment
↓
Your Backend
↓
verify signature
↓
check event/idempotency
↓
update payment
↓
confirm order / enqueue work
```

12. Webhook Security

- Verify webhook signature according to provider documentation.
- Do not trust arbitrary client-provided payment status.
- Validate event type and required identifiers.
- Protect against replay using event IDs/timestamps/idempotency where supported.
- Follow the provider's required acknowledgement semantics.

13. Webhook Idempotency

Webhook event:
evt_123

First delivery:
→ process
→ mark event processed

Same event delivered again:
evt_123
→ already processed
→ don't apply business effect twice

Webhook providers can retry delivery. Isliye webhook handling bhi idempotent honi chahiye.

14. Gateway Timeout

```
Your server → Gateway
                ↓
                timeout
```

Question:
Did payment happen?

Answer:
UNKNOWN

Do NOT blindly:
→ mark FAILED
→ create another charge

Timeout ka matlab payment definitely failed nahi hota. Gateway status query, webhook, reconciliation ya provider-specific verification flow use karna pad sakta hai.

15. Unknown Payment State

REQUEST_SENT

↓
TIMEOUT
↓
UNKNOWN

Possible actual state:
SUCCESS / FAILED / PENDING

Need:
→ status verification
→ webhook
→ reconciliation

16. Database Transaction

Payment state update aur related business state changes ko carefully transactional boundaries mein design karo. External gateway call ko normal DB transaction ke andar long time hold karna usually undesirable hota hai.

```
DB:
create payment PENDING
commit
```

```
External gateway:
customer pays
```

```
Webhook:
DB transaction:
  verify event
  update payment SUCCESS
  update order appropriately
commit
```

17. Don't Keep DB Transaction Open During Gateway Call

Gateway response seconds tak wait kar sakta hai. Is duration mein DB transaction open rakhna locks/connections unnecessarily consume kar sakta hai. Durable state machine + short DB transactions use karo.

18. Payment Record

```
Payment
{
  id,
  orderId,
  userId,
  amount,
  currency,
  status,
  gateway,
  gatewayPaymentId,
  idempotencyKey,
  createdAt,
  updatedAt
}
```

19. Amount Integrity

Final amount browser se blindly trust nahi karna chahiye. Server ko order/cart/pricing data se payable amount calculate or validate karna chahiye.

```
Client says:
amount = ₹10
```

```
Server:
cart total = ₹1,000
```

```
Server must not trust ₹10
without validating business rules.
```

20. Currency

Amount ke saath currency store karo. Multi-currency systems mein exchange rate, rounding and settlement rules explicitly define karo.

21. Double Charge Prevention

Potential duplicate paths:

1. user double-click
2. client retry
3. API retry
4. worker retry
5. webhook retry
6. gateway timeout + retry

Defense:

- idempotency
- unique constraints
- state transitions
- provider transaction ID
- reconciliation

22. State Transition Protection

PENDING → SUCCESS ■

PENDING → FAILED ■

SUCCESS → PENDING ■

FAILED → SUCCESS ? depends on business/provider flow

Use controlled transitions.

State machine ko database/application layer mein enforce karna useful hai, especially concurrent webhook/API updates ke cases mein.

23. Concurrency

Webhook aur status API same payment ko simultaneously update kar sakte hain. Row-level locking, optimistic concurrency/versioning, conditional updates ya idempotent state transitions use kiye ja sakte hain.

24. Unique Constraints

Useful uniqueness examples:

gatewayPaymentId
providerEventId
idempotencyKey + merchant/user scope

Purpose:

database itself helps prevent duplicates.

25. Refund

SUCCESS



Refund Request



REFUND_PENDING



Gateway



REFUNDED

Refund ko original payment se linked separate record/entity ke roop mein model karna useful ho sakta hai, especially partial/multiple refunds ke cases mein.

26. Partial Refund

Payment = ■1,000

Refund 1 = ■200

Refund 2 = ■300

Total refunded = ■500

Remaining refundable = ■500

Total refund amount ko original captured amount se exceed nahi hone dena chahiye, subject to provider/business rules.

27. Ledger Concept

Financial systems mein ledger ka purpose monetary movements ka durable, auditable record maintain karna hai. Payment status table aur accounting ledger same thing nahi hote.

Payment:
PAY-1001 = SUCCESS

Ledger:
Customer/merchant accounts
debit/credit entries
with immutable transaction references

Production financial systems mein accounting correctness ke liye append-oriented/auditable ledger patterns common hain; exact accounting design domain-specific hota hai.

28. Reconciliation

Reconciliation ka matlab internal payment records ko gateway/provider settlement or transaction reports ke saath compare karna. Ye un cases ko detect karta hai jahan webhook miss hua, timeout hua, ya internal state mismatch ho gayi.

Internal DB
↓ compare
Gateway report
↓
Matched / Mismatch

Mismatch:
→ investigate
→ status correction
→ financial operations

29. Why Reconciliation Is Important

Distributed systems network ya integration failures se immune nahi hote. Reconciliation ek safety net hai jo asynchronous inconsistencies ko detect karne mein help karta hai.

30. Payment Polling

Agar provider webhook available/usable nahi hai, backend provider status API ko poll kar sakta hai. Polling ko controlled intervals, timeout, retry and terminal-state rules ke saath design karo.

31. Queue in Payment System

Payment Service
↓
Queue
■→ webhook processing
■→ reconciliation
■→ receipt/email
■→ fraud checks
■→ analytics

Critical payment state should not depend blindly on an async side job.

32. Outbox Pattern

Agar payment DB update ke baad event publish karna hai, DB update successful aur queue publish failed hone ka gap aa sakta hai. Outbox pattern event ko same DB transaction mein store karke later reliable publisher se publish kar sakta hai.

```
DB Transaction
■→ payment = SUCCESS
■→ outbox_event = PAYMENT_SUCCESS

Commit
↓
Outbox Worker
↓
Queue/Event Bus
```

33. Security

- Use TLS for network communication.
- Never store sensitive card data unless your architecture is explicitly designed and compliant for it.
- Prefer provider-hosted/SDK/tokenized flows where appropriate.
- Verify webhook signatures.
- Authenticate and authorize payment APIs.
- Validate server-side amount/order state.
- Use idempotency keys.
- Audit important state changes.
- Apply least-privilege access to payment data and credentials.

34. PCI / Compliance Awareness

Payment architecture mein applicable regulatory and payment-network requirements matter karte hain. PCI DSS scope ko reduce karne ke liye tokenization/hosted payment flows commonly considered hote hain, but exact compliance responsibility architecture aur provider par depend karti hai.

35. Scaling Payment Service

```
Clients
↓
Load Balancer
↓
Payment API × N
↓
DB
■→ Payment records
■→ Idempotency
■→ Outbox
↓
Queue
↓
Workers
■→ notifications
■→ reconciliation
■→ async processing

External Gateway
↑↓
Payment Service
```

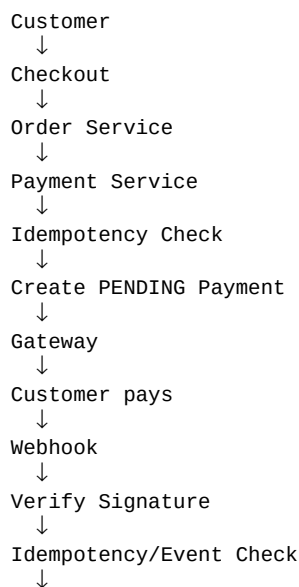
36. Observability

- Payment attempts/sec
- Success/failure rate
- Pending/unknown payments
- Gateway latency
- Gateway timeout rate
- Webhook latency
- Webhook failure/retry count
- Duplicate/idempotency hits
- Refund success/failure
- Reconciliation mismatches
- Queue depth/consumer lag
- p50/p95/p99 API latency

37. Production Failure Scenarios

Failure	Handling
User double-click	Backend idempotency
API timeout	Do not blindly retry charge; verify existing payment
Gateway timeout	Mark state appropriately; verify later
Webhook duplicate	Provider event ID/idempotency
Webhook missed	Reconciliation/status verification
DB update race	Transactions/locking/conditional state transitions
Worker duplicate	Idempotent job processing
Gateway outage	Controlled failure and retry; avoid unsafe duplicate attempts
Refund retry	Idempotent refund operation

38. End-to-End Production Flow



DB Transaction

■→ Payment = SUCCESS

■→ Order = CONFIRMED

■→ Outbox Event



Queue



Notifications



Reconciliation runs periodically

39. Interview Q&A;

Question	Short answer
Payment system ka biggest challenge?	Distributed failures and duplicate financial effects ko safely handle karna.
Idempotency kya hai?	Same logical request repeat hone par duplicate charge ke bina same logical result/effect.
Webhook kyun?	Provider asynchronous payment state changes merchant backend ko notify kar sakta hai.
Webhook duplicate?	Event ID/idempotency se logically once process karo.
Gateway timeout ka meaning?	Payment state unknown ho sakti hai; blindly FAILED ya retry charge nahi.
Order vs payment?	Separate entities; one order can have multiple payment attempts.
Why reconciliation?	Internal and gateway records ke mismatch detect/correct karne ke liye.
Refund?	Successful payment ke against money reversal operation.
Partial refund?	Original captured amount ka subset refund karna.
Ledger?	Auditable monetary movement record; payment status table se distinct.
Outbox?	DB change aur event publication ke reliability gap ko reduce karta hai.
How prevent duplicate charge?	Idempotency + unique constraints + provider transaction reference + safe state transitions.
Why not long DB transaction around gateway?	External call slow/unpredictable; DB resources unnecessarily hold honge.

40. Practice Scenarios

1. User ne Pay button 3 baar click kiya. Design a safe payment API.
2. Payment gateway request timeout ho gayi. Kya payment retry karoge?
3. Gateway ne SUCCESS webhook 3 baar bheja. System kya kare?
4. Payment SUCCESS hai but order still PENDING hai. Root cause aur recovery design karo.
5. DB says FAILED but gateway says SUCCESS. Kaise detect aur correct karoge?
6. Design partial refunds.
7. Gateway 5 minutes down hai. New payment attempts ka behavior kya hoga?
8. Design reconciliation for 10 million payments/day.
9. Payment worker crash after gateway success but before DB update. What happens?
10. Design idempotency for a payment API.
11. Explain why client-side button disabling is not enough.
12. Design payment metrics and Grafana dashboard.

41. Local Node.js Practice

Node.js + Express + MySQL/PostgreSQL + Redis + BullMQ use karke fake payment gateway banao. Payment API mein idempotency implement karo, PENDING/SUCCESS/FAILED states banao, fake gateway timeout simulate karo, webhook endpoint implement karo, duplicate webhook handle karo, refund flow add karo, outbox + worker implement karo aur reconciliation job chalao.

Phase 1:
POST /orders
POST /payments

Phase 2:
Idempotency

Phase 3:
Fake Gateway
→ success/failure/timeout

Phase 4:
Webhook
→ signature + idempotency

Phase 5:
Refund

Phase 6:
Outbox + BullMQ

Phase 7:
Reconciliation

Phase 8:
Prometheus + Grafana

42. Final Cheat Sheet

PAYMENT SYSTEM

Order
↓
Payment Attempt
↓
Idempotency
↓
PENDING
↓
Gateway
↓
Webhook / Status Verification
↓
SUCCESS / FAILED / UNKNOWN
↓
Order State Update
↓
Outbox → Queue → Async Work

Must handle:
→ duplicate requests
→ duplicate webhooks
→ timeout / UNKNOWN
→ retries
→ state transitions
→ refunds
→ reconciliation
→ ledger/audit
→ security
→ observability

Golden interview line:
"I'll separate Order and Payment, use backend idempotency to prevent duplicate charges, maintain

a payment state machine, treat gateway timeout as potentially UNKNOWN, verify webhooks, make webhook processing idempotent, use short DB transactions, and run reconciliation as a safety net."